

Hands on Experience the Spring Boot Application Development, Test and Fixing Vulnerabilities:

Task 01: Building a simple CRUD Application with Spring Boot and MySQL

1. Set up a Spring Boot project using the Spring Initializer website or your preferred IDE.

Use following dependencies,

- **Project:** Maven
- **Language:** Java
- **Spring Boot:** Latest stable version
- **Dependencies:**
 - Spring Web
 - Spring Data JPA
 - MySQL Driver
 - Spring Boot DevTools (optional)

2. Implement an Employee Management System for ABC Company Pvt Ltd and following functions shall be included,

- Adding a new employee record
- Update Existing employee record
- Delete records of any employee who is resigned from the company
- List all records of an employee
- List a specific record as per the requirement

3. Configure MySQL Database

- Setup ER diagram to build connectivity between entities
- Setup at least two entities (Ex -: Employee data, Department Data, Project Details etc.)

4. Configure credentials and other config data on an application.property or application.yaml file.

3. Create a simple entity class to represent the data that will be stored in the database (e.g. EmployeeMangement, DepartmentandProjects etc.)

4. Use the Spring Data JPA to connect to a MySQL database and perform CRUD operations (Create, Read, Update, Delete) on the entity classes. ((Try to use clean code architecture for setup classes))
5. Create a REST endpoint for each CRUD operation, such as:
 - POST endpoint for creating new records
 - GET endpoint for retrieving a list of all records
 - GET endpoint for retrieving a single record by id
 - PUT endpoint for updating a record
 - DELETE endpoint for deleting a record
6. Implement proper error handling for invalid requests, such as requests with missing required parameters or attempts to update non-existent records.
7. Test your endpoints using tools such as Postman or curl.
8. Develop Test cases for main methods in your service classes. (use required dependencies to write unit tests)

Task 02: Build a Jar File and Scan and generate a Sonar Report using SonarQube

1. Add relevant sonar dependencies to pom.
2. Install Linux on your machine using appropriate VM
3. Install SonarScanner on the Linux Server (Please check on the prerequisites to install Sonar)
4. Create sonar-project.properties in the project root on Linux:
5. Build your Spring Boot project on Linux (with Maven or Gradle):
6. Run SonarScanner on Linux inside the project folder:
7. **Check results on SonarQube:**
 - Open browser: <http://<linux-server-ip>:9000>
 - Login → Project → Dashboard → See bugs, code smells, vulnerabilities, and coverage.